Microsoft

Building RAG with PostgreSQL

Pamela Fox, Python Cloud Advocate, @pamelafox
Joshua Johnson, Azure Database for PostgreSQL Product Manager

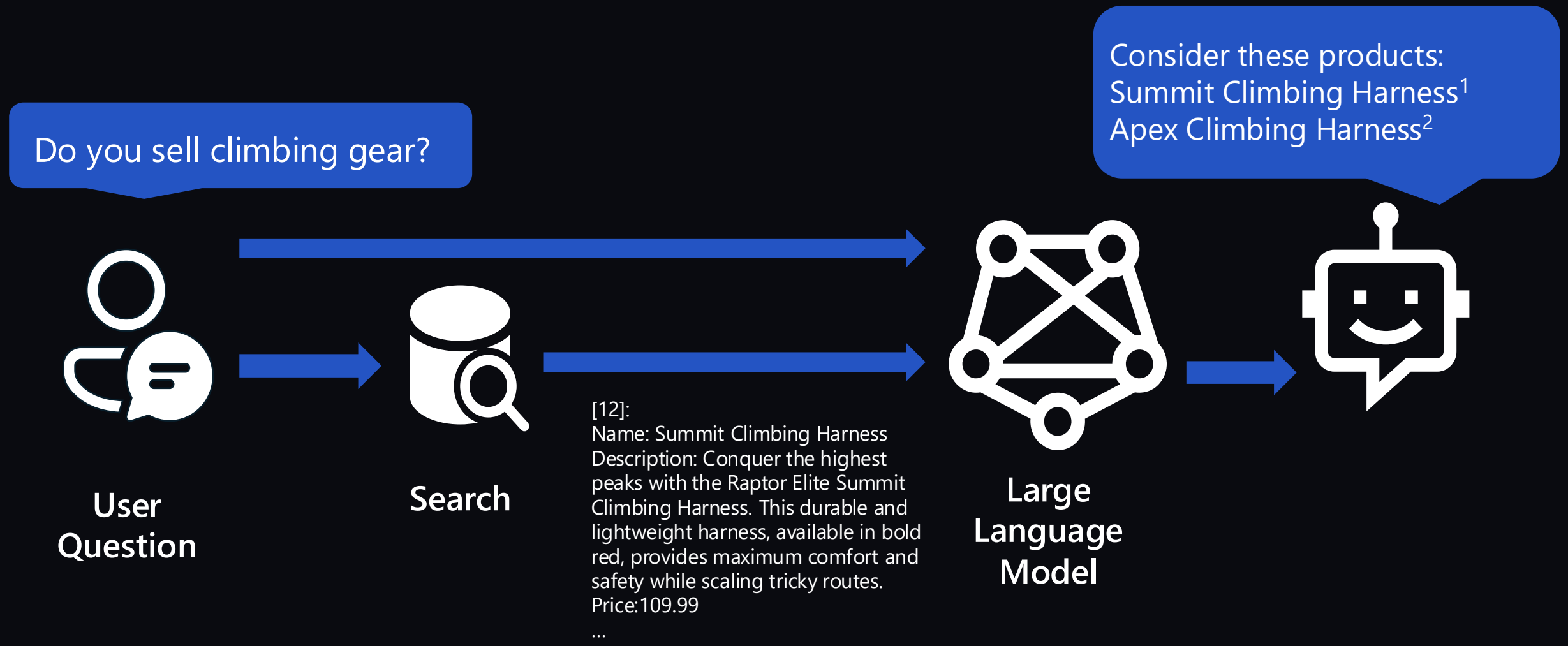
Agenda

- Simple RAG with PostgreSQL
- RAG with Azure PostgreSQL solution
- AI extensions for Azure PostgreSQL

Microsoft

RAG with PostgreSQL: Getting started

Simple RAG steps

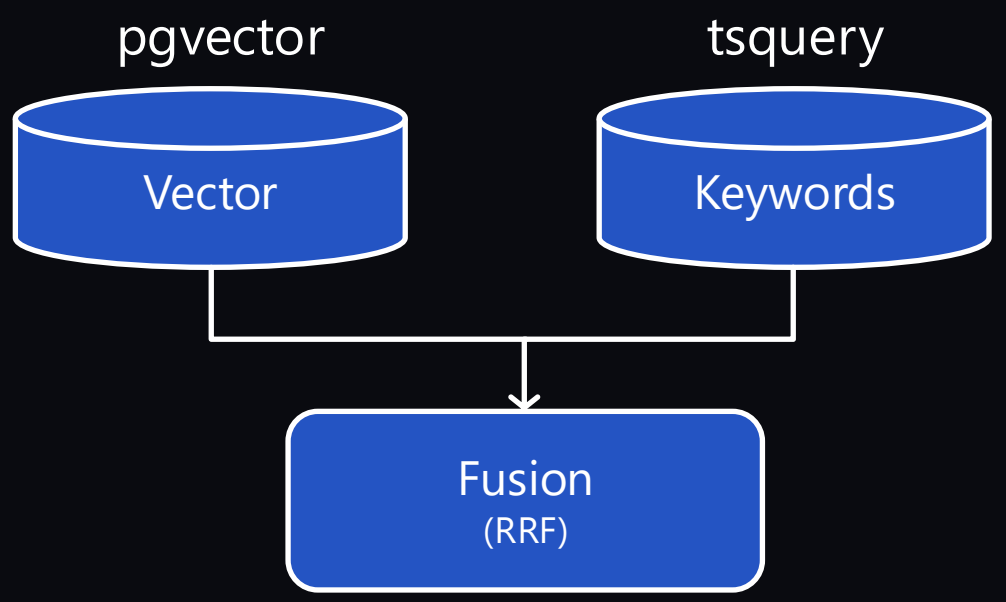


RAG search step: Go hybrid!

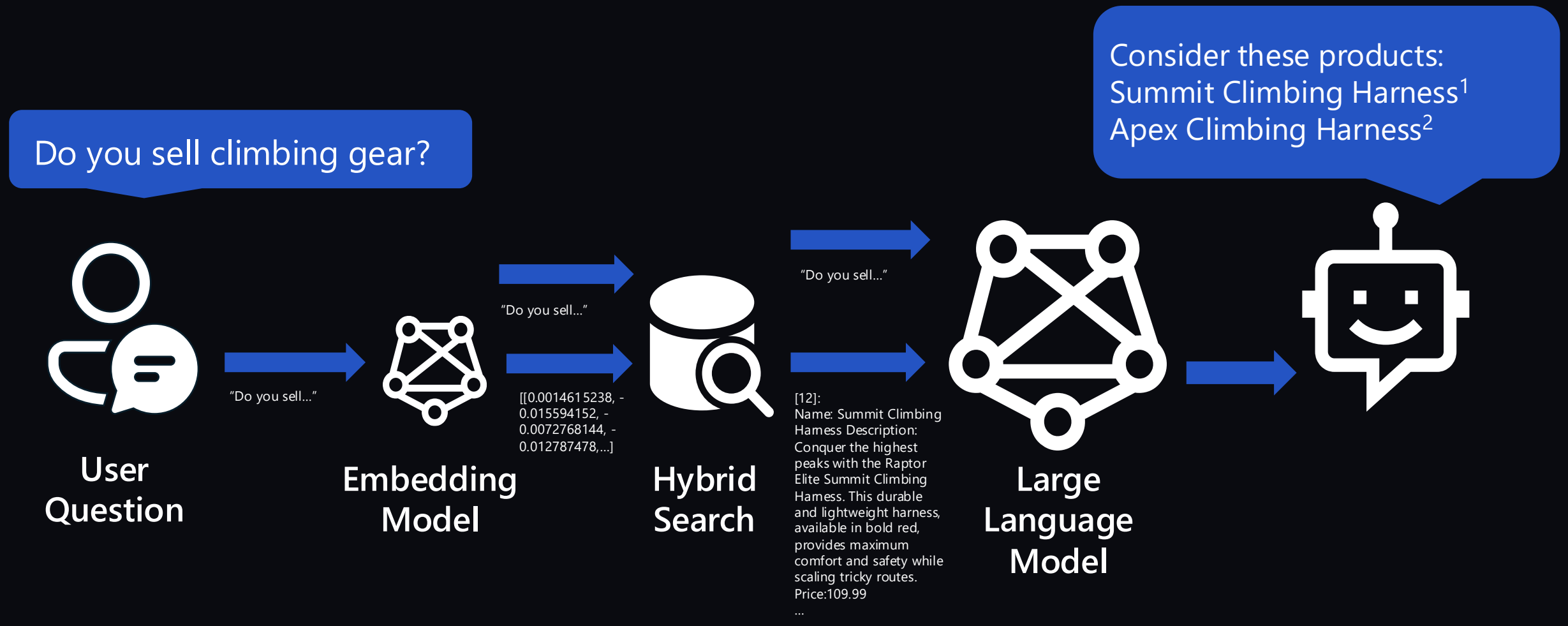
Vector search is best for finding semantically related matches

Keyword search is best for exact matches (proper nouns, numbers, etc)

Hybrid search combines vector search and keyword search, optimally using Reciprocal-Rank-Fusion for merging results and a ML model to re-rank results after



Simple RAG *with hybrid search*



Setting up pgvector

Install the pgvector extension:

github.com/pgvector/pgvector

Enable extension:

```
CREATE EXTENSION IF NOT EXISTS vector;
```

Tip: Try the extension in the pgvector dev container:
github.com/pamelafox/pgvector-playground

Adding a vector column

Choose a vector embedding model

Add a vector embedding column:

```
ALTER TABLE videos ADD COLUMN embedding vector(256)
```

Add an index for more efficient searching:

```
CREATE INDEX ON videos USING hnsw (embedding vector_ip_ops)
```

Creating an index for query efficiency

Use either HNSW or IVFflat:

```
CREATE INDEX ON videos USING hnsw (embedding vector_ip_ops)
```

Use inner product (vector_ip_ops) on normalized embeddings.

Use cosine product (vector_cosine_ops) otherwise.

Computing vector embeddings

Python:

```
string_to_embed = row[0] + " " + row[1]
response = client.embeddings.create(
    model="text-embedding-3-small",
    input=string_to_embed,
    dimensions=256
)
embedding = response.data[0].embedding

cur.execute("UPDATE videos SET embedding = %s WHERE id = %s",
(np.array(embedding), row[0]))
```

Vector search SQL query

```
SELECT id
  FROM videos
  ORDER BY embedding <=> %(embedding)s
  LIMIT 20
```

The operator should match the operator used in the index creation.

Use inner product <#> on normalized embeddings.

Use cosine product <=> otherwise.

Full-text search SQL query

```
SELECT id FROM videos, plainto_tsquery('english', %(query)s) query
  WHERE to_tsvector('english', description) @@ query
  ORDER BY ts_rank_cd(to_tsvector('english', description), query)
  DESC LIMIT 20
```

Hybrid search query

```
WITH semantic_search AS (
        SELECT id, RANK () OVER (ORDER BY embedding <=> %(embedding)s) AS rank
        FROM videos
        ORDER BY embedding <=> %(embedding)s
        LIMIT 20
),
keyword_search AS (
        SELECT id, RANK () OVER (ORDER BY ts_rank_cd(to_tsvector('english', description), query) DESC)
        FROM videos, plainto_tsquery('english', %(query)s) query
        WHERE to_tsvector('english', description) @@ query
        ORDER BY ts_rank_cd(to_tsvector('english', description), query) DESC
        LIMIT 20
)
SELECT
COALESCE(semantic_search.id, keyword_search.id) AS id,
COALESCE(1.0 / (%(k)s + semantic_search.rank), 0.0) +
COALESCE(1.0 / (%(k)s + keyword_search.rank), 0.0) AS score
FROM semantic_search
FULL OUTER JOIN keyword_search ON semantic_search.id = keyword_search.id
ORDER BY score DESC
LIMIT 5
```

https://github.com/pgvector/pgvector-python/blob/master/examples/hybrid_search_rrf.py

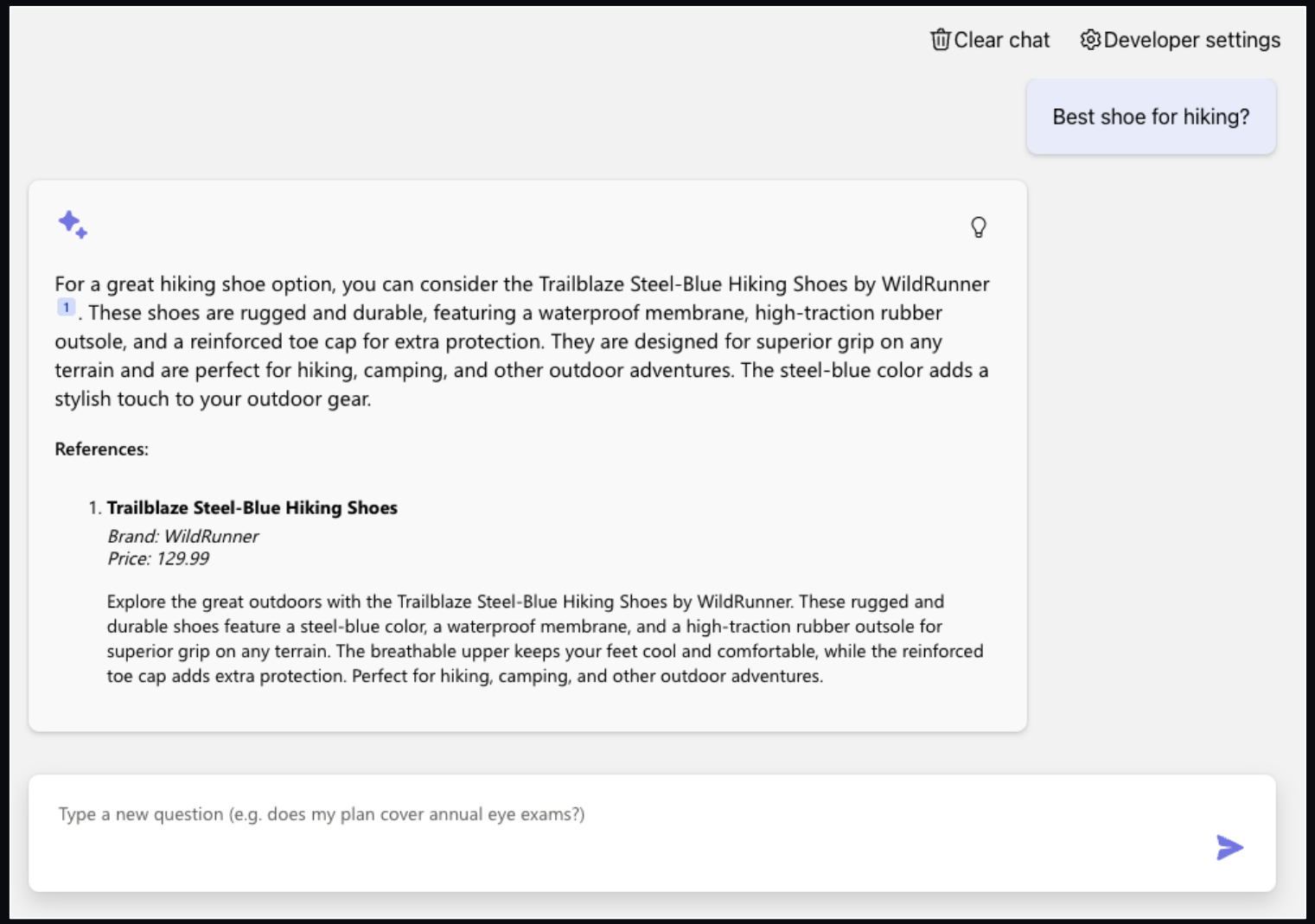
Orchestrating a RAG flow with Python

```
question = "any videos on python?"

cur.execute("SELECT ... ")
results = cur.fetchall()
for result in results:
  formatted_results += f"## {result[1]}\n\n{result[2]}\n"

response = client.complete(
  messages=[
    SystemMessage(content="Answer question according to sources..."),
    UserMessage(content=question +
              "\n\nSources:\n\n" + formatted_results)],
model="gpt-4o",
temperature=0.3)
```

Microsoft

Full RAG with PostgreSQL solution

Simple RAG (on PostgreSQL)



Azure OpenAI +
Azure PostgreSQL Flexible Server +
Azure Container Apps

Code:
aka.ms/rag-postgres

Demo:
aka.ms/rag-postgres/demo

Code walkthrough

Typescript frontend

(React, FluentUI)

chat.tsx
`makeApiRequest()`

→

api.ts
`chatApi()`

Python backend

(FastAPI, Uvicorn)

app.py
`chat()`

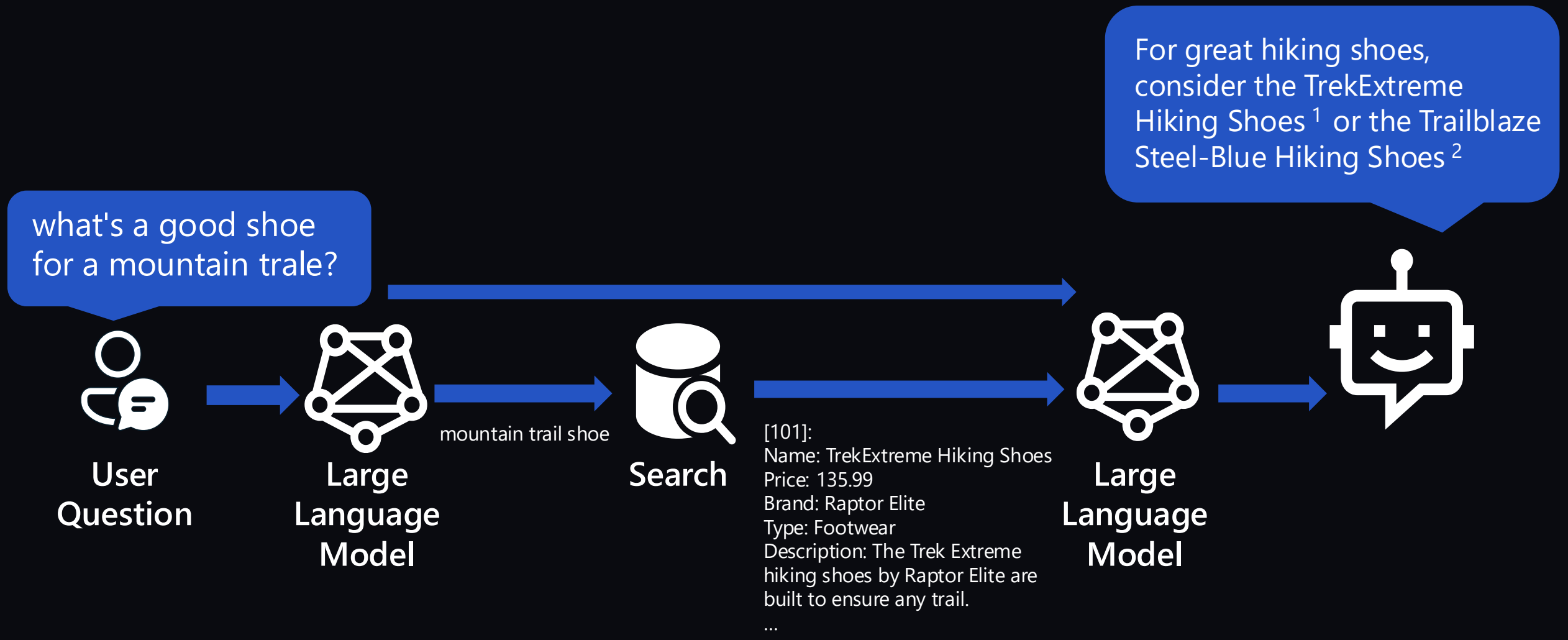
→

simple_rag.py
`run()`

↓

```
get_search_query()
compute_text_embedding()
search()
get_messages_from_history()
chat.completions.create()
```

Advanced RAG with query re-writing



Advanced RAG (on PostgreSQL)

Climbing gear cheaper than $30?

Sure, we have a climbing gear item that is cheaper than $30:
- [1]: Guardian Blue Chalk Bag by Legend for $21.99

References:

1. **Guardian Blue Chalk Bag**
   Brand: Legend
   Price: 21.99

   Stay focused on your route with the Guardian Blue Chalk Bag by Legend. This durable bag features a spacious compartment for your chalk, a drawstring closure, and a waist belt for easy access while climbing. The vibrant blue color adds a stylish touch to your climbing gear.

Type a new question (e.g. does my plan cover annual eye exams?)

Thought process

Prompt to generate search arguments

model: "gpt-35-turbo" | deployment: "gpt-35-turbo"

```
[
  "{'role': 'system', 'content': 'Below is a history of the conversation so far, and a new question asked by the user that needs to be answered by searching database rows.\\nYou have access to an Azure PostgreSQL database with an items table that has columns for title, description, brand, price, and type.\\nGenerate a search query based on the conversation and the new question.\\nIf the question is not in English, translate the question to English before generating the search query.\\nIf you cannot generate a search query, return the original user question.\\nDO NOT return anything besides the query.'}",
  "{'role': 'user', 'content': 'Climbing gear cheaper than $30?'}"
]
```

Search using generated search arguments

top: 3 | vector_search: true | text_search: true | filters: [{"column":"price","comparison_operator":"<","value":30}]

climbing gear

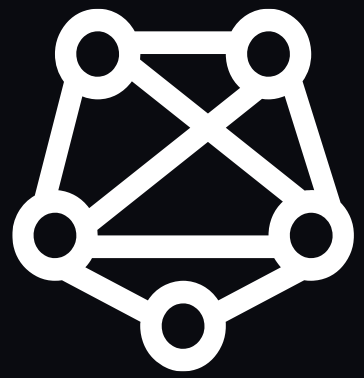
Search results

```
[
  {
    "id": 31,
    "type": "Climbing",
    "brand": "Legend",
    "name": "Guardian Blue Chalk Bag",
```

Query rewriting with function calling

Do you sell climbing gear cheaper than $30?

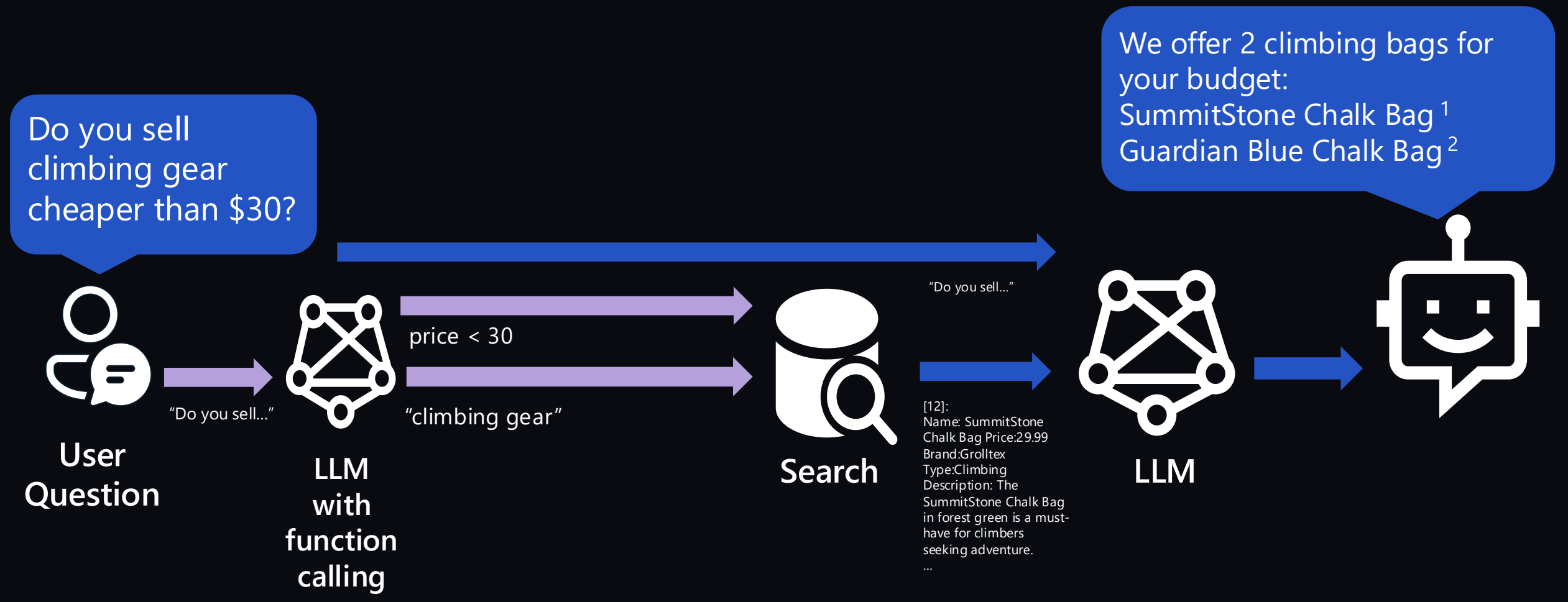
User Question



Large Language Model with function calling

```
search_database(
  "climbing_gear",
  {"column": "price",
   "operator" : "<",
   "value" : "30"
  }
)
```

Advanced RAG with query rewriting via function calling



Code walkthrough

Typescript frontend

(React, FluentUI)

chat.tsx
`makeApiRequest()`

→

api.ts
`chatApi()`

Python backend

(FastAPI, Uvicorn)

app.py
`chat()`

→

advanced_rag.py
`run()`

↓

```
get_search_query()
compute_text_embedding()
search()
get_messages_from_history()
chat.completions.create()
```

Deploying with the Azure Developer CLI

Login to your Azure account:

```
azd auth login --use-device-code
```

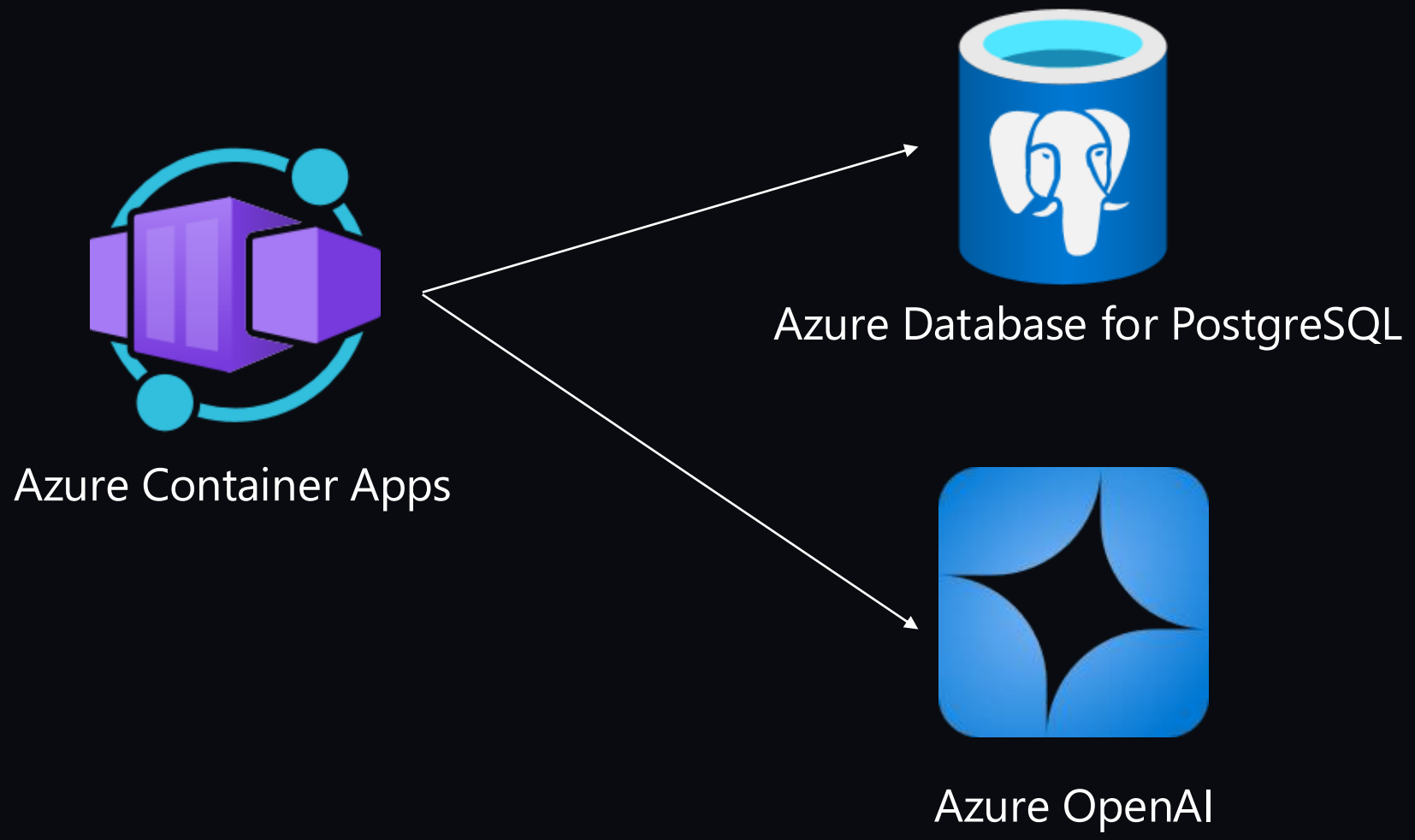
Create a new azd environment: (to track deployment parameters)

```
azd env new
```

Provision resources and deploy app:

```
azd up
```

Application architecture on Azure



Microsoft

AI extensions for Azure Database for PostgreSQL

Azure Database for PostgreSQL – AI features

Vector data type

pgvector extension

Native vector data type – store embeddings

Vector indexing for performant searches

Efficient ANN searches within the DB

Azure AI extension

SQL Interface to Azure OpenAI

Create embeddings from SQL Statements

SQL interface to Azure AI Language services

Invoke Azure ML models from within the DB

Integrations

LangChain

Semantic Kernel

LlamaIndex

AI Services integrated into Azure Postgres

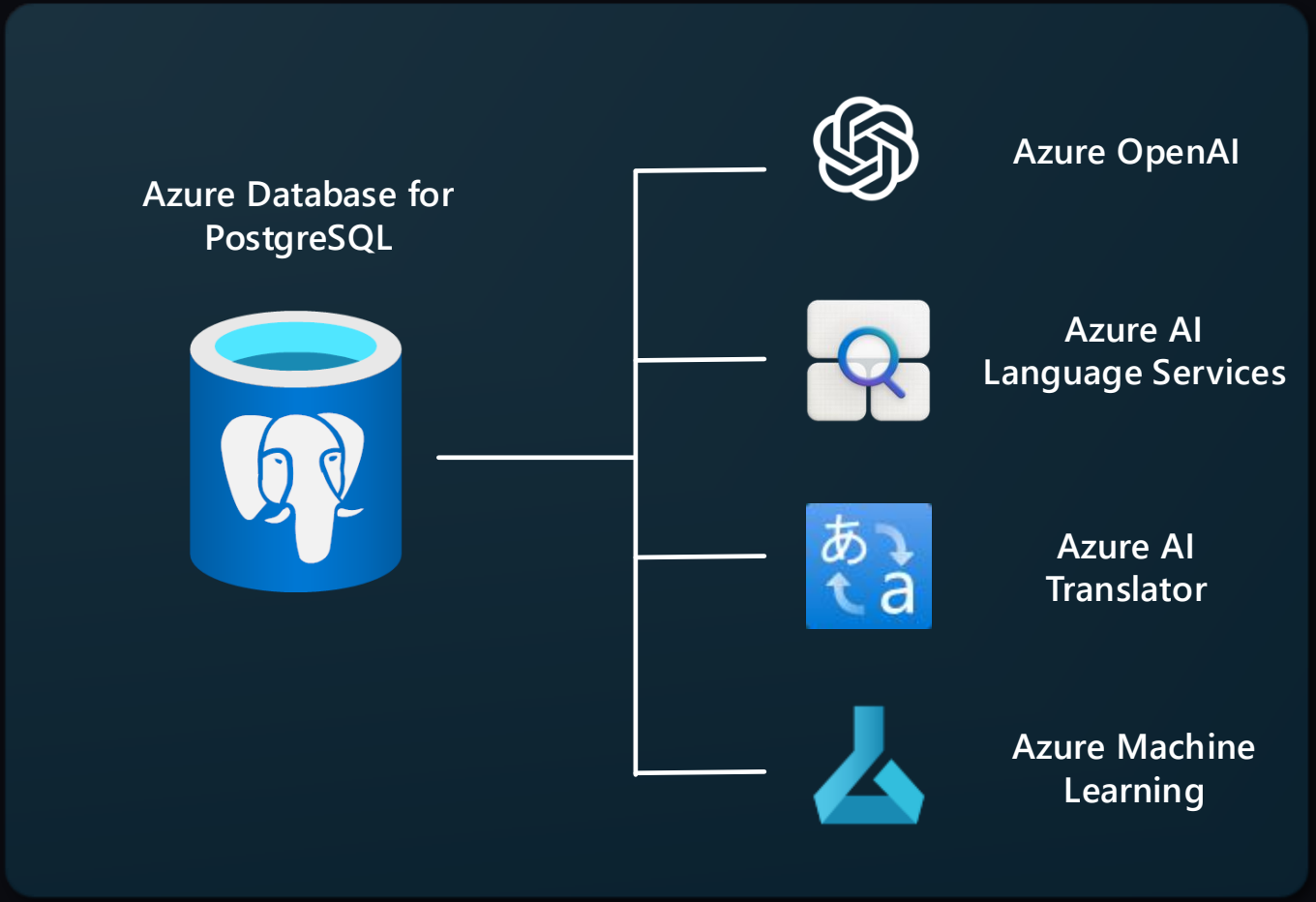
Make remote calls directly from PostgreSQL

azure_ai extension

Exceptional simplicity out of the box

- Azure OpenAI
- Azure AI Language Services
- Azure AI Translator
- Azure Machine Learning

Enables developers to rapidly adopt new AI capabilities in their solution without complex re-architecture or refactoring

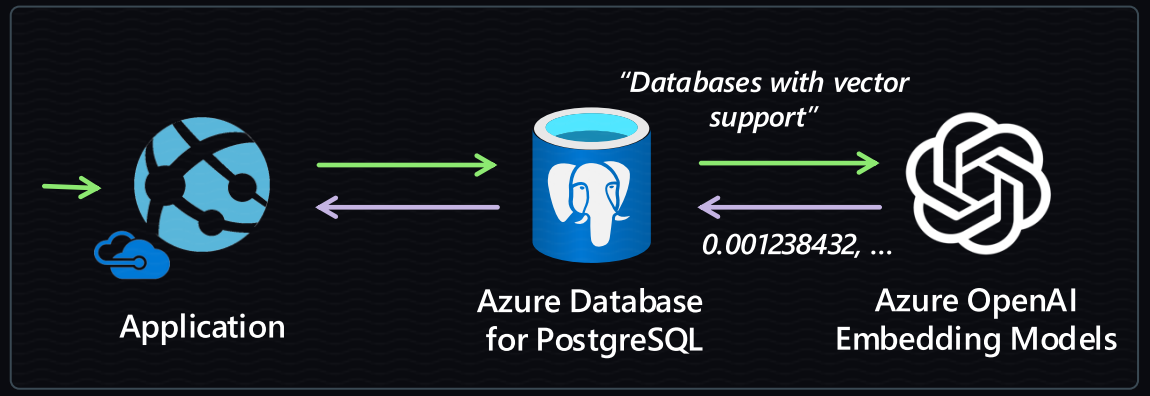


Vector Generation

Unique Remote + In-Database Embedding Models

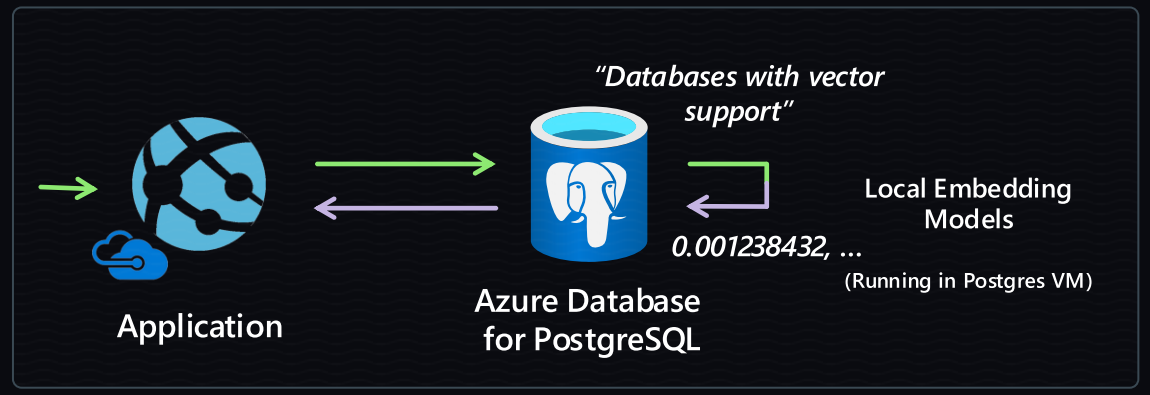
Remote Embedding Models

```
SELECT * FROM <table>
ORDER BY
database_description <->
azure_openai.create_embeddings(
'text-embedding-ada-002',
'Databases with vector support')
```



In-Database Embedding Models
(Preview)

```
SELECT * FROM <table>
ORDER BY
recipe_embedding <#>
azure_local_ai.create_embeddings(
'multilingual-e5-small:v1',
'Databases with vector support')
```



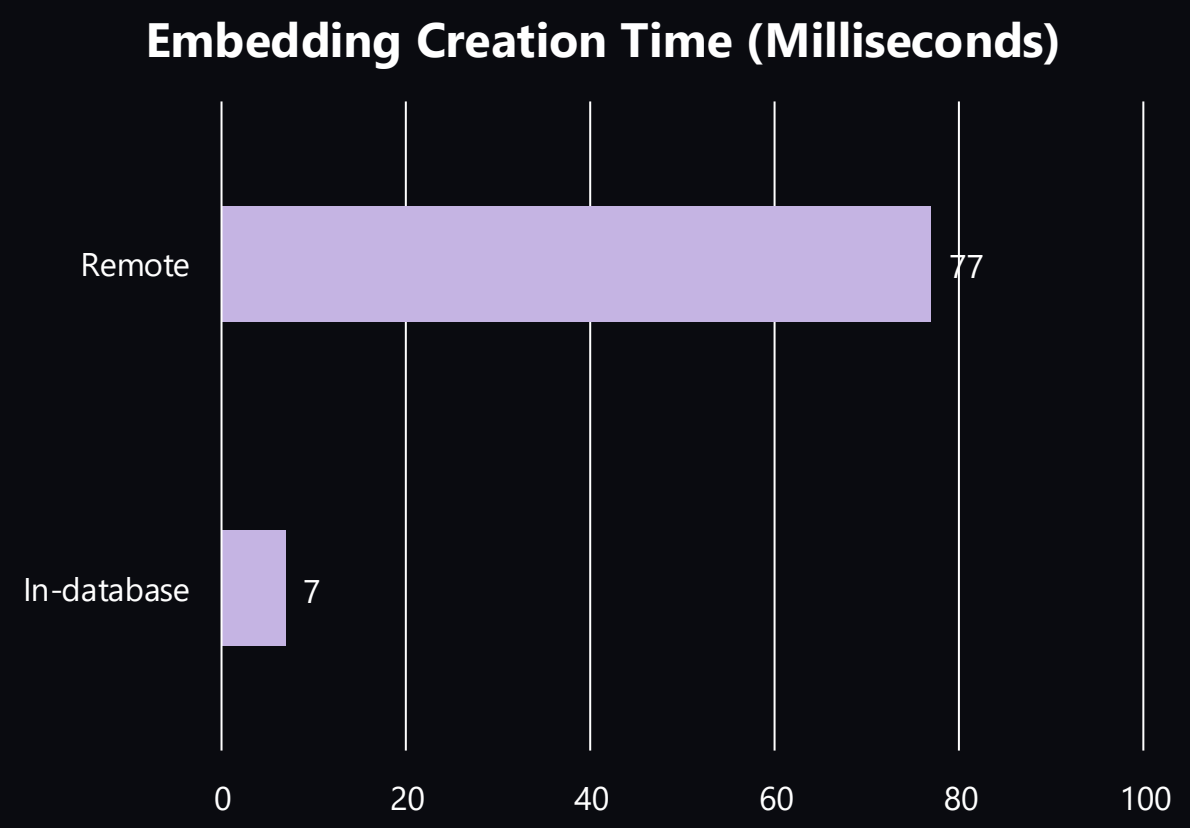
In-Database Embedding Models

Low-latency embedding creation for OLTP workloads

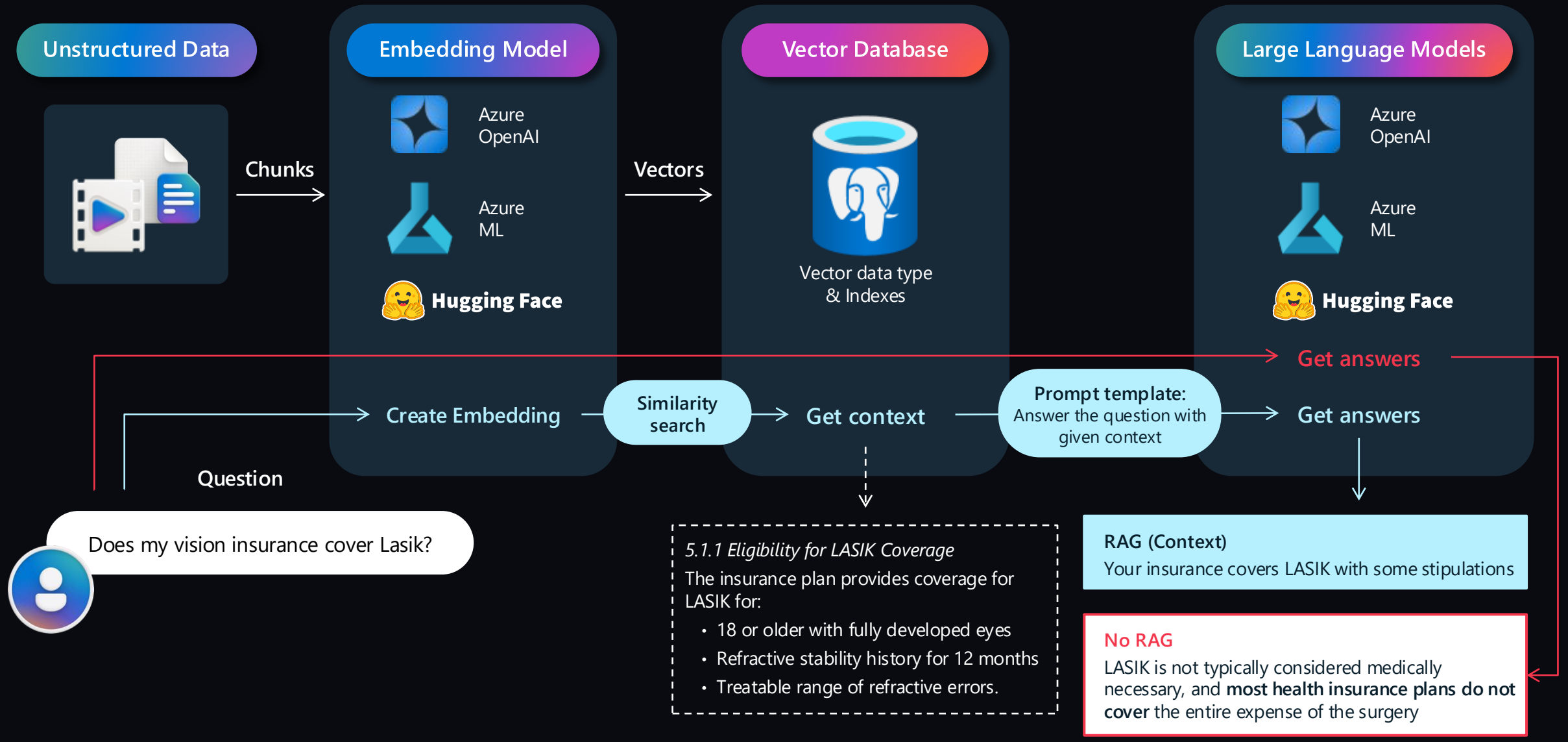
Data Privacy, data doesn’t leave database

Lower latency and higher throughput

Cost predictability



RAG Pattern with Azure Database for PostgreSQL



eShop – Powered by Azure DB for PostgreSQL

eShop with chat assistant

.Net Aspire

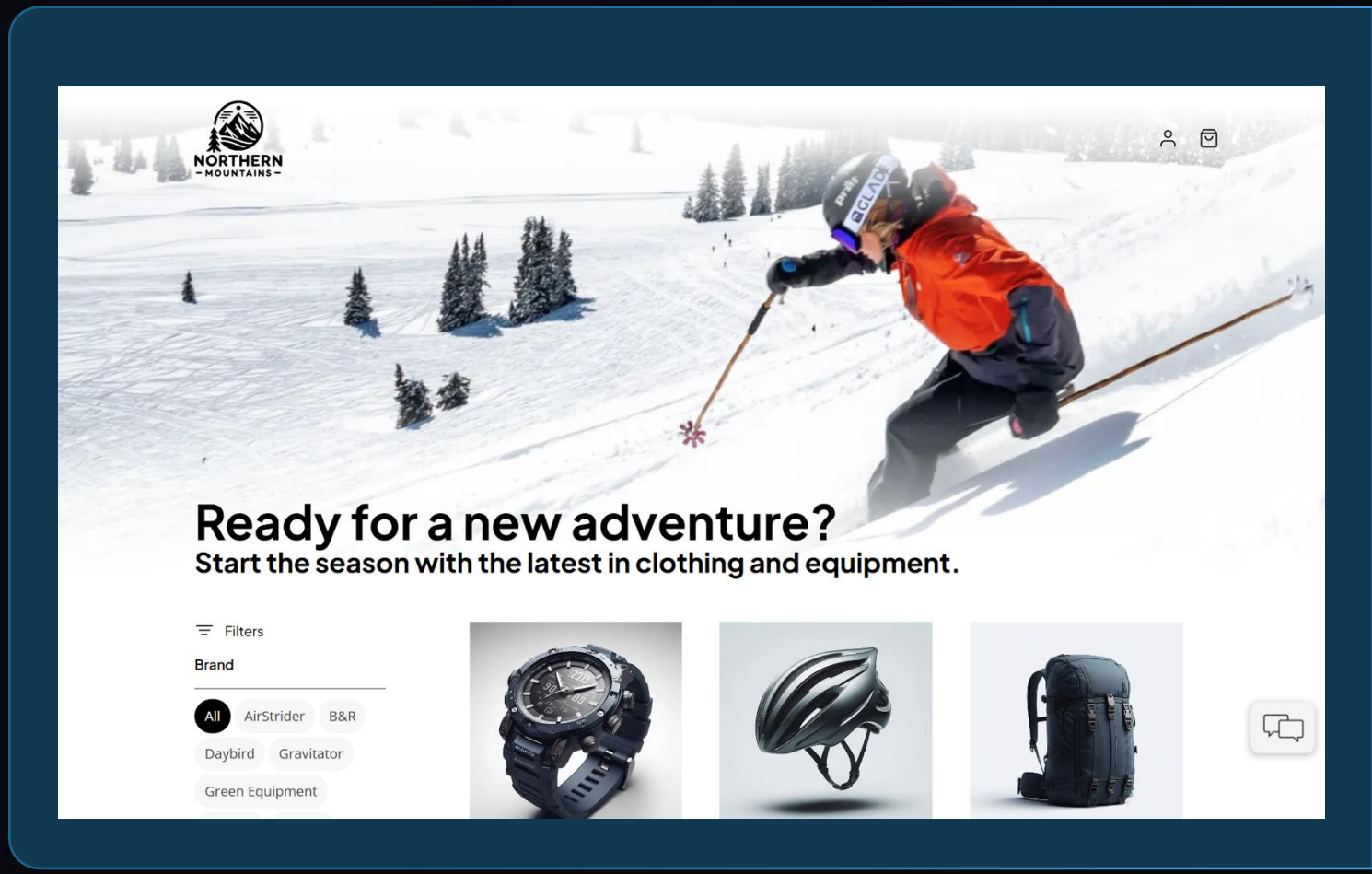
Entity Framework Core

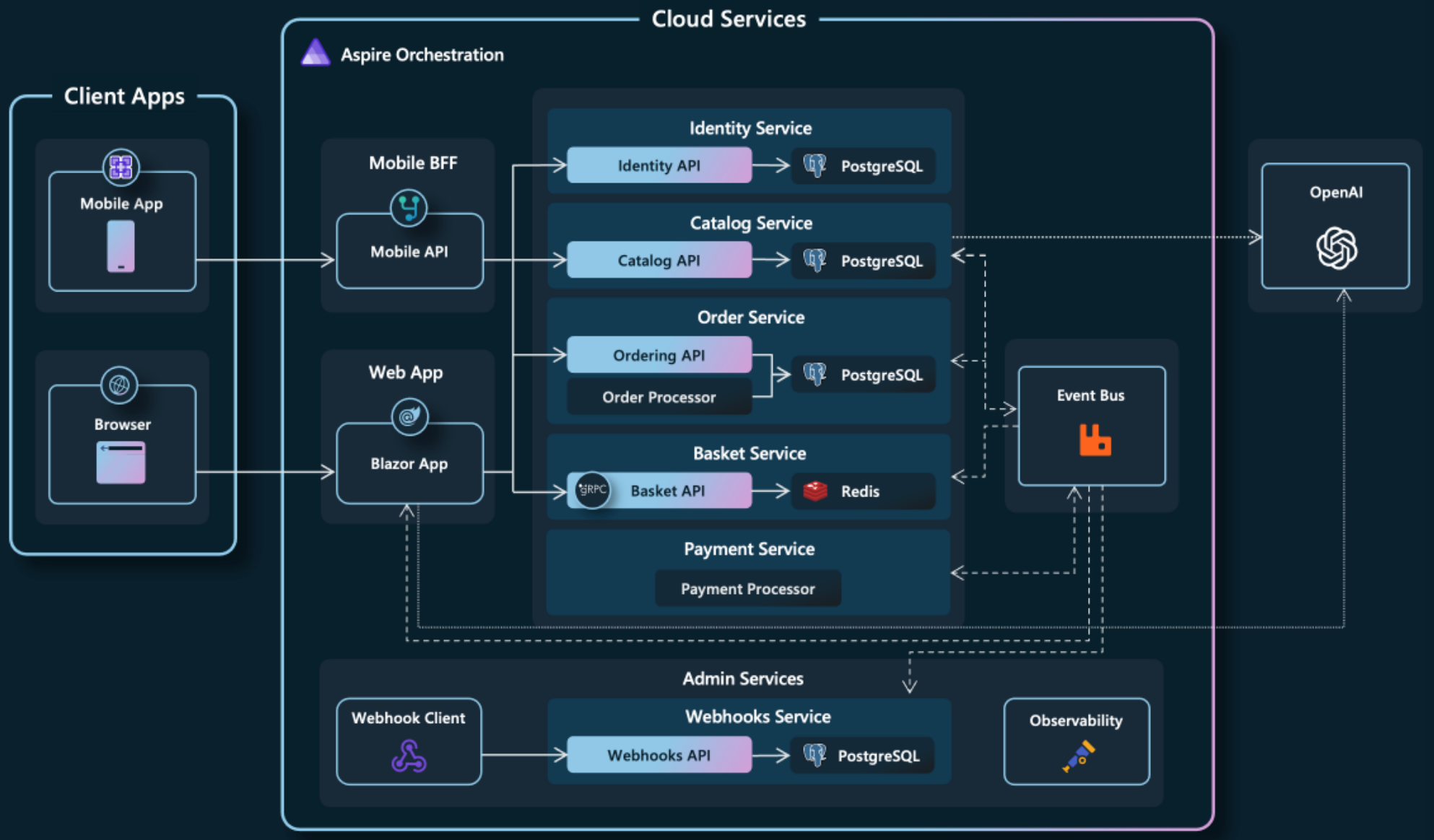
Semantic Kernel and Azure OpenAI

Azure DB for PostgreSQL

Extensions:

- azure_ai
- pgvector





Azure-Samples/eShopOnAzure

NORTHERN
MOUNTAINS

Ready for a new adventure?

Start the season with the latest in clothing and equipment.

Filters

Brand

All | AirStrider | B&R | Daybird | Gravitator | Green Equipment

rg-eshoponazure_new

Resource group

- Search
- Overview
- Activity log
- Access control (IAM)
- Tags
- Resource visualizer
- Events
- Settings
- Cost Management
- Monitoring
- Automation
- Help

Create | Manage view | Delete resource group | Refresh | Export to CSV | Open query | Assign tags | Move | Delete | Group by none

Essentials | View Cost | JSON View

Resources | Recommendations (1)

Filter for any field... | Type equals all | Location equals all | Add filter

Name ↑	Type	Location
acrk6f6mqzr5pgna	Container registry	East US
appinsightsk6f6mqzr5pgna	Application Insights	East US
basket-api	Container App	East US
cae-k6f6mqzr5pgna	Container Apps Environment	East US
catalog-api	Container App	East US
eventbusk6f6mqzr5pgna	Service Bus Namespace	East US
Failure Anomalies - appinsightsk6f6mqzr5pgna	Smart detector alert rule	Global
kv4a4e4f18k6f6mqzr5pgna	Key vault	East US
law-k6f6mqzr5pgna	Log Analytics workspace	East US
mi-k6f6mqzr5pgna	Managed Identity	East US

Showing 1 - 10 of 19 results.

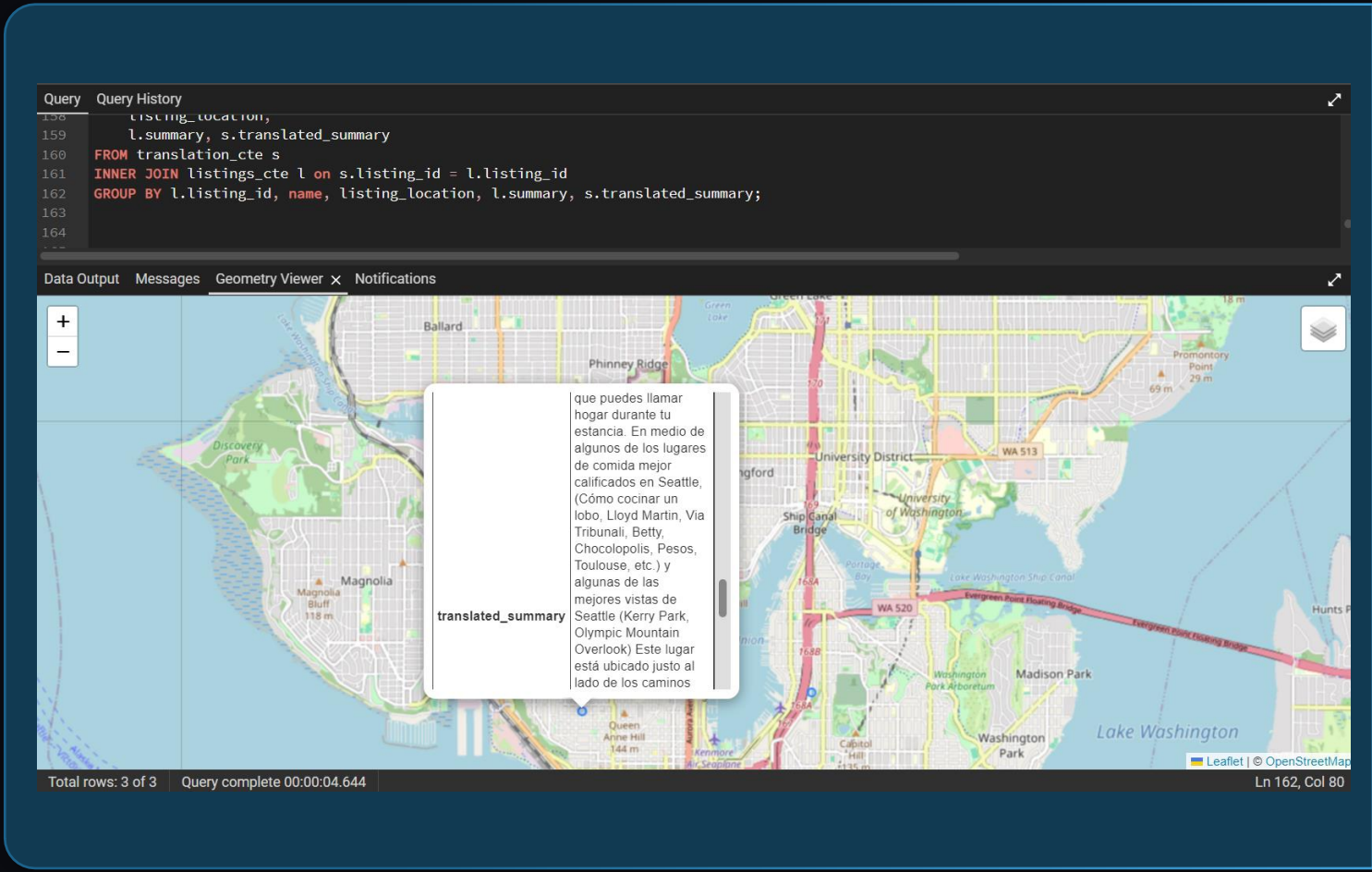
Give feedback

Real-time translation of listing summaries

Hybrid search with real-time translation.

Combining,

- Vector search
- Geospatial data
- Azure AI Services



pgAdmin 4

File Object Tools Help

plorer

Servers (10)

pgai-postgis-senti... | pgai-translation-de... | pgsql-demo2.sql | 4th demo set - advanced query with translation.sql*

pgailocaljj1/azure_pg_admin@pgai-local-jj1

No limit

Query | Query History

```sql
-- Hybrid search with real time translation. Combining pgvector, postgis, azure_local_ai, azure_ai, Azure Language Services
WITH listings_cte AS (
    SELECT l.listing_id, name, listing_location, summary FROM listings l
    INNER JOIN calendar c ON l.listing_id = c.listing_id
    WHERE ST_DWithin(
        listing_location::geography,
        ST_GeographyFromText('POINT(-122.349358 47.620422)')
        ,3218 )
    AND c.date = '2016-01-13'
    AND c.available = 't'
    AND c.price <= 75.00
    AND l.listing_id IN (SELECT listing_id FROM listings)
    ORDER BY local_description_vector <=> azure_local_ai.create_embeddings('multilingual-e5-small:v1',
       'properties in Seattle near the Space Needle with 3 bedrooms, and allow pets')::vector
    LIMIT 5
)
SELECT
    l.listing_id,
    name,
    listing_location,
    l.summary,
```

Data Output | Messages | Notifications

	review_summary text[]
1	{"The document is a series of reviews from various individuals who have stayed at Paula and John's home in Seattle. They praise the home, its location near Lake Union and downtown, and the hosts' hospitality and helpfulness. The revie

Total rows: 1 of 1 | Query complete 00:00:04.472 | Ln 35, Col 1

pgAdmin 4

File Object Tools Help

plorer

Servers (10)

pgai-postgis-senti... | pgai-translation-de... | pgsql-demo2.sql | 4th demo set - advanced query with translation.sql*

pgailocaljj1/azure_pg_admin@pgai-local-jj1

No limit

Query | Query History

```sql
-- Hybrid search with real time translation. Combining pgvector, postgis, azure_local_ai, azure_ai, Azure Language Services
WITH listings_cte AS (
    SELECT l.listing_id, name, listing_location, summary FROM listings l
    INNER JOIN calendar c ON l.listing_id = c.listing_id
    WHERE ST_DWithin(
        listing_location::geography,
        ST_GeographyFromText('POINT(-122.349358 47.620422)')
        ,3218 )
    AND c.date = '2016-01-13'
    AND c.available = 't'
    AND c.price <= 75.00
    AND l.listing_id IN (SELECT listing_id FROM listings)
    ORDER BY local_description_vector <=> azure_local_ai.create_embeddings('multilingual-e5-small:v1',
       'properties in Seattle near the Space Needle with 3 bedrooms, and allow pets')::vector
    LIMIT 5
)
SELECT
    l.listing_id,
    name,
    listing_location,
    l.summary,
```

Data Output | Messages | Geometry Viewer | Notifications

	listing_id integer	name character varying (50)	listing_location geometry	summary character varying (2000)
1	3697351	5 Blocks from Pike Place Market	0101000020E610000087F9F202EC955EC098FA795391CE4740	Located in the heart of downtown squarely between the Space Needle and Pike Place Market, this large 3 bedr
2	4848255	Private room near downtown Seat...	0101000020E610000066F7E461A1945EC01A69A9BC1DD14740	Large apartment with queen size beds, over sized spare closet. Full sunlight in the living and dining area. Resid
3	6961346	Bright clean special Jan/Feb prices	0101000020E610000049A297512C975EC0F25EB532E1CF4740	Walk to the best spots Seattle has to offer from this centrally located studio apartment in the heart of the Histo
4	5900224	Walk to the best spots in Seattle	0101000020E610000060CD018239965EC08ACDC7B5A1CE4740	Hello, I have a top floor furnished 3 bedroom unit in the heart of Capitol Hill; an urban residential area situated
5	6389657	Wonderful Seattle Penthouse	0101000020E610000017D4B7CCE9965EC09AB1683A3BD14740	We love our penthouse! Guests will have access to the basement that holds the washer and dryer. I will be avai

Total rows: 5 of 5 | Query complete 00:00:02.528 | Ln 76, Col 7